

Credit Card Approval Prediction using Machine Learning

Abstract:

Credit Card Approval Prediction is a machine learning-based web application that automates the credit card approval process by predicting whether an applicant is eligible for a credit card based on financial and demographic information. The system analyzes applicant details such as income, employment status, credit history, family size, and other relevant attributes to provide an instant approval or rejection prediction. Multiple machine learning models are compared to select the best-performing model, which is integrated into a Flask web application and deployed using IBM Watson Machine Learning for real-time predictions.

Problem Statement:

Banks receive thousands of credit card applications every day. Manual verification of applicant details is time-consuming, prone to human error, and delays the approval process. The objective of this project is to automate credit card approval prediction using machine learning to improve speed, accuracy, and decision-making.

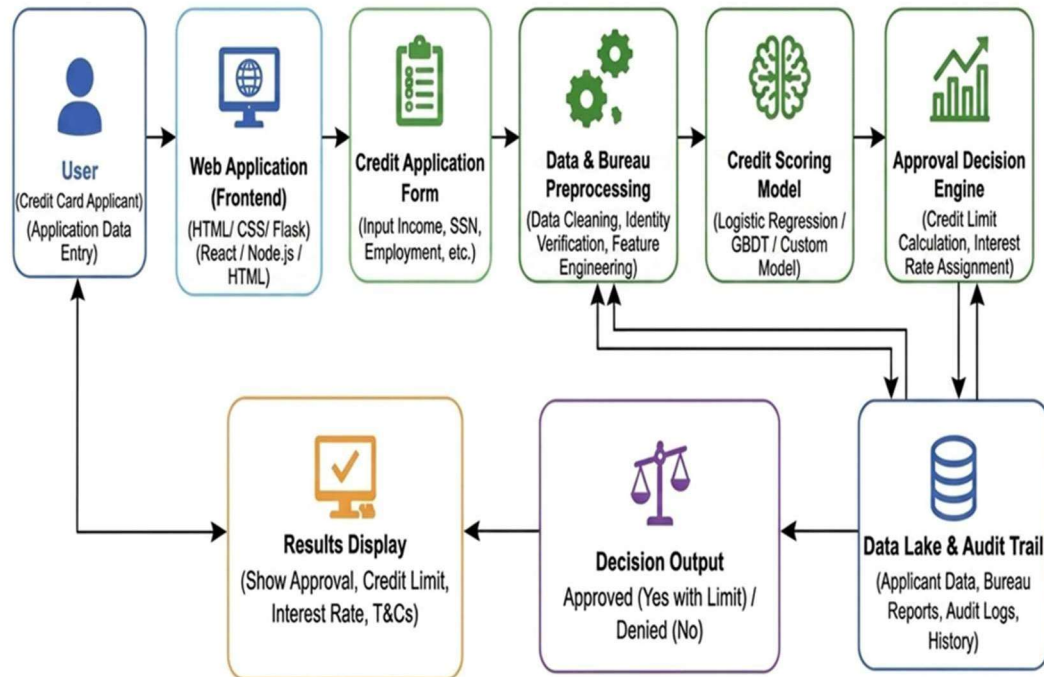
Scenarios

Scenario 1: A bank employee enters an applicant's financial and personal details into the web application. The trained machine learning model instantly predicts whether the application is likely to be approved or rejected, helping the bank process applications more efficiently.

Scenario 2: A financial institution screens applicants with poor credit history or previous loan defaults. The system identifies high-risk applicants using historical data and machine learning models, reducing financial risk.

Scenario 3: A customer checks their eligibility before submitting a credit card application. The application predicts the likelihood of approval, helping customers avoid unnecessary rejections.

Technical Architecture:



Description:

The system follows a three-layer architecture consisting of a Frontend, Flask Backend, and Machine Learning Prediction Engine.

- The Frontend provides a simple interface for entering applicant details.
- The Flask Backend validates user input, preprocesses applicant data, and communicates with the trained machine learning model.
- The Machine Learning Engine predicts whether the application will be approved or rejected using the trained model.
- The prediction result is displayed instantly through the web interface and can also be accessed through IBM Watson Machine Learning cloud deployment.

Tech Stack :

Python Programming
 Vs code
 Flask Framework
 HTML, CSS, Bootstrap
 Pandas
 NumPy
 Scikit-learn
 XGBoost
 IBM Watson Machine Learning

Installation Steps:

```
</> Bash

git clone <repository_url>

cd CreditCardApprovalPrediction

python -m venv venv

venv\Scripts\activate

pip install -r requirements.txt

python app.py
```

Open your browser:

```
http://127.0.0.1:5000
```

Usage Instructions

1. Open the web application.
2. Enter applicant details such as gender, income, employment status, education, family members, housing type, and credit history.
3. Click the Predict button.
4. The machine learning model processes the information.
5. The application displays whether the credit card application is Approved or Rejected

Project Workflow:

Milestone 1: Data Collection & Preprocessing

Collect Credit Card Approval Dataset
Handle Missing Values
Encode Categorical Features
Feature Selection
Split Dataset into Training and Testing

Milestone 2 : Machine Learning Model Development

Train Logistic Regression
Train Decision Tree
Train Random Forest
Train XGBoost
Compare Accuracy
Select Best Model

Milestone 3: Flask Application Development

Create Home Page
Develop Prediction Form
Load Trained Model (.pkl)
Predict Credit Card Approval
Display Prediction Result

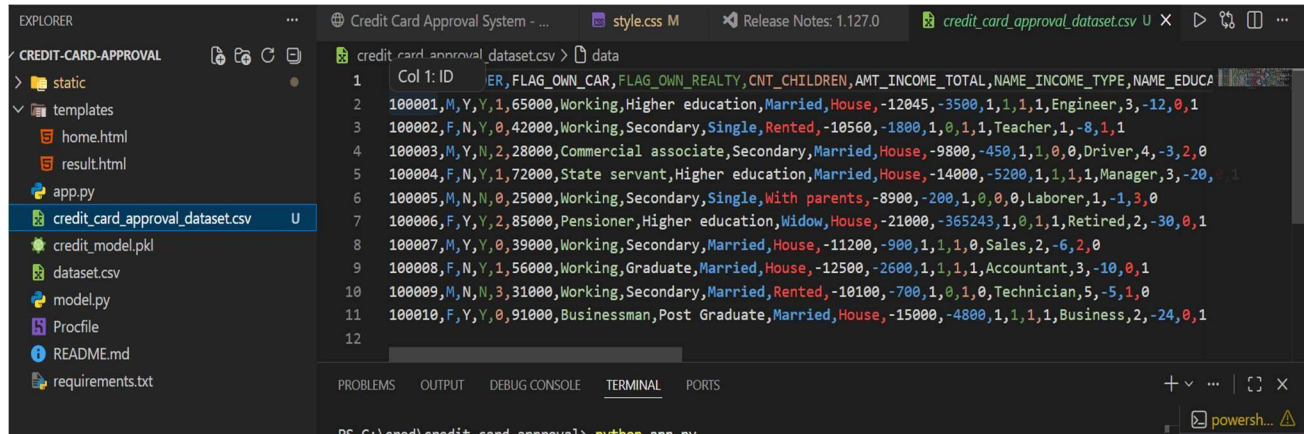
Milestone 4: Frontend Development

- Responsive User Interface
- Applicant Information Form
- Result Display Page
- Input Validation
- User-Friendly Design

Milestone 5: Conclusion

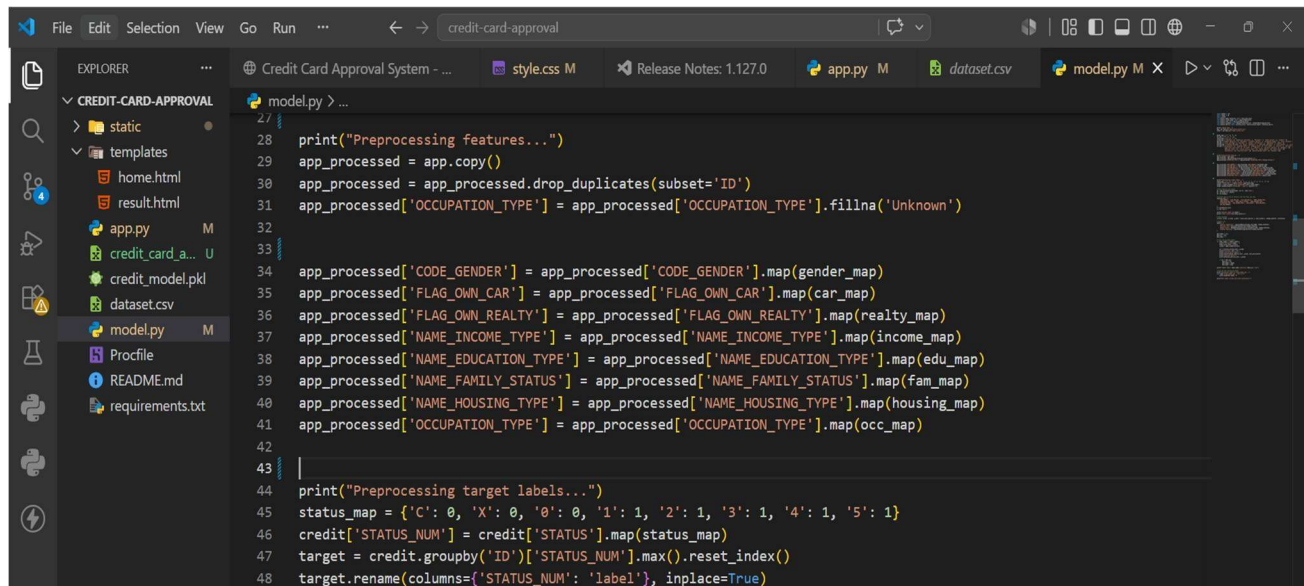
Evaluate Model Performance
Compare Machine Learning Algorithms
Document Results
Complete Project Deployment

Milestone 1: Data Collection & Preprocessing



```

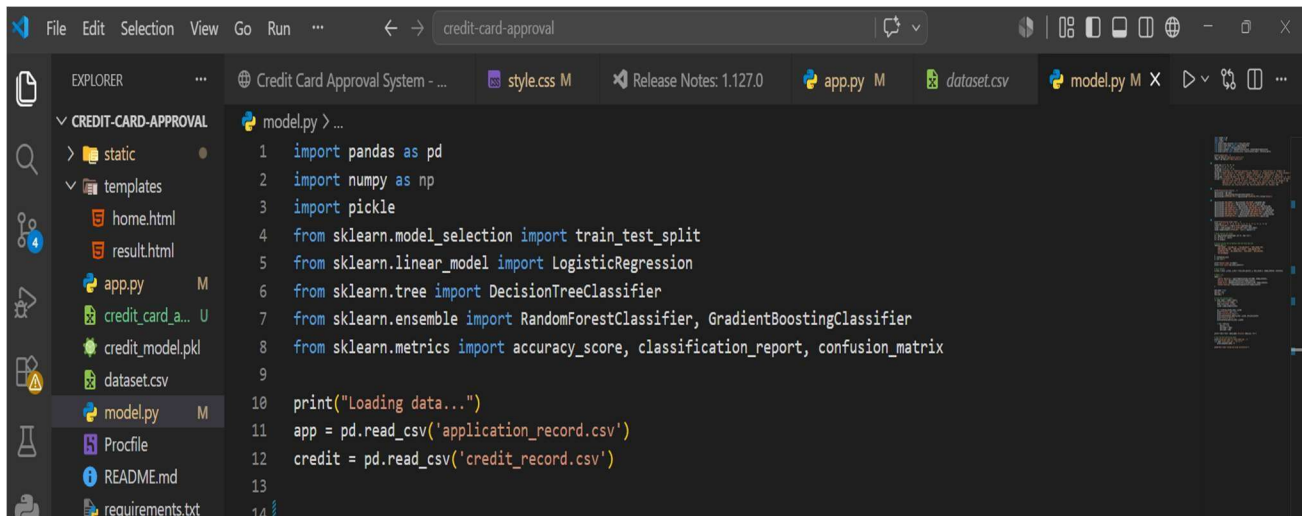
1  Col 1: ID      GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, CNT_CHILDREN, AMT_INCOME_TOTAL, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE, STATUS_NUM
2  100001,M,Y,Y,1,65000,Working,Higher education,Married,House,-12045,-3500,1,1,1,1,Engineer,3,-12,0,1
3  100002,F,N,Y,0,42000,Working,Secondary,Single,Rented,-10560,-1800,1,0,1,1,Teacher,1,-8,1,1
4  100003,M,Y,N,2,28000,Commercial associate,Secondary,Married,House,-9800,-450,1,1,0,0,Driver,4,-3,2,0
5  100004,F,N,Y,1,72000,State servant,Higher education,Married,House,-14000,-5200,1,1,1,1,Manager,3,-20,1
6  100005,M,N,N,0,25000,Working,Secondary,Single,With parents,-8900,-200,1,0,0,0,Laborer,1,-1,3,0
7  100006,F,Y,Y,2,85000,Pensioner,Higher education,Widow,House,-21000,-365243,1,0,1,1,Retired,2,-30,0,1
8  100007,M,Y,Y,0,39000,Working,Secondary,Married,House,-11200,-900,1,1,1,0,Sales,2,-6,2,0
9  100008,F,N,Y,1,56000,Working,Graduate,Married,House,-12500,-2600,1,1,1,1,Accountant,3,-10,0,1
10 100009,M,N,N,3,31000,Working,Secondary,Married,Rented,-10100,-700,1,0,1,0,Technician,5,-5,1,0
11 100010,F,Y,Y,0,91000,Businessman,Post Graduate,Married,House,-15000,-4800,1,1,1,1,Business,2,-24,0,1
12
  
```



```

27
28 print("Preprocessing features...")
29 app_processed = app.copy()
30 app_processed = app_processed.drop_duplicates(subset='ID')
31 app_processed['OCCUPATION_TYPE'] = app_processed['OCCUPATION_TYPE'].fillna('Unknown')
32
33
34 app_processed['CODE_GENDER'] = app_processed['CODE_GENDER'].map(gender_map)
35 app_processed['FLAG_OWN_CAR'] = app_processed['FLAG_OWN_CAR'].map(car_map)
36 app_processed['FLAG_OWN_REALTY'] = app_processed['FLAG_OWN_REALTY'].map(realty_map)
37 app_processed['NAME_INCOME_TYPE'] = app_processed['NAME_INCOME_TYPE'].map(income_map)
38 app_processed['NAME_EDUCATION_TYPE'] = app_processed['NAME_EDUCATION_TYPE'].map(edu_map)
39 app_processed['NAME_FAMILY_STATUS'] = app_processed['NAME_FAMILY_STATUS'].map(fam_map)
40 app_processed['NAME_HOUSING_TYPE'] = app_processed['NAME_HOUSING_TYPE'].map(housing_map)
41 app_processed['OCCUPATION_TYPE'] = app_processed['OCCUPATION_TYPE'].map(occ_map)
42
43
44 print("Preprocessing target labels...")
45 status_map = {'C': 0, 'X': 0, '0': 0, '1': 1, '2': 1, '3': 1, '4': 1, '5': 1}
46 credit['STATUS_NUM'] = credit['STATUS'].map(status_map)
47 target = credit.groupby('ID')['STATUS_NUM'].max().reset_index()
48 target.rename(columns={'STATUS_NUM': 'label'}, inplace=True)
  
```

Milestone 2 : Machine Learning Model Development



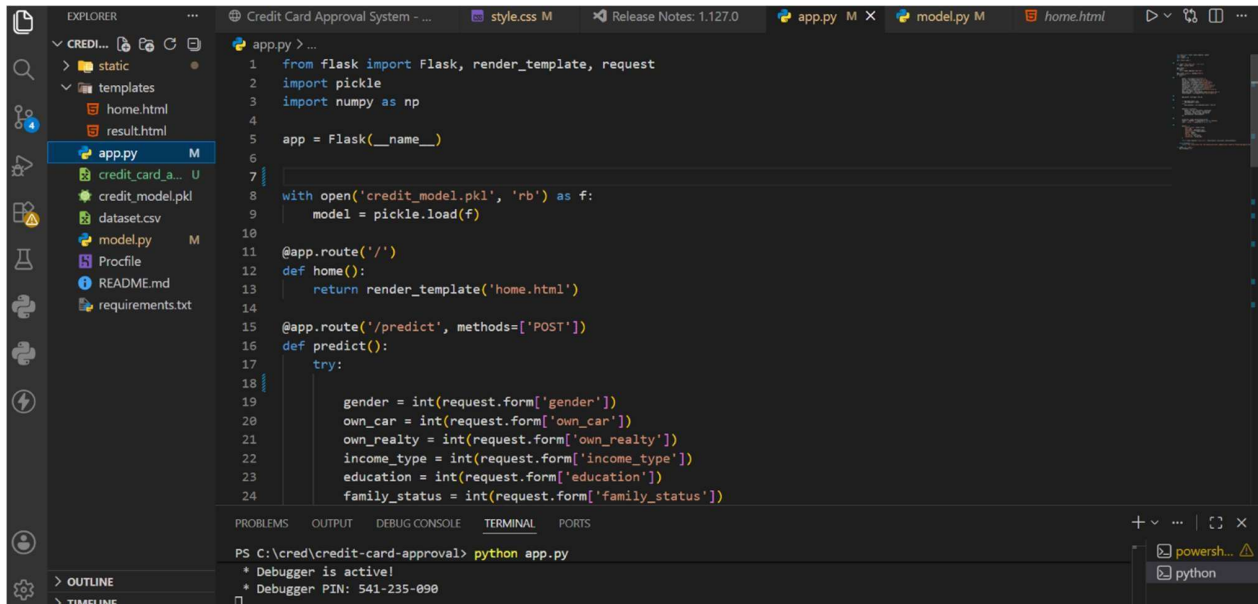
The screenshot shows a Visual Studio Code editor with a project named "Credit Card Approval System". The Explorer sidebar on the left shows the project structure, including a "static" folder, "templates" folder, and files like "home.html", "result.html", "app.py", "credit_card_a...", "credit_model.pkl", "dataset.csv", "model.py", "Profile", "README.md", and "requirements.txt". The main editor window displays the code in "model.py":

```

1  import pandas as pd
2  import numpy as np
3  import pickle
4  from sklearn.model_selection import train_test_split
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.tree import DecisionTreeClassifier
7  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
8  from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
9
10 print("Loading data...")
11 app = pd.read_csv('application_record.csv')
12 credit = pd.read_csv('credit_record.csv')
13
14

```

Milestone 3: Flask Application Development



The screenshot shows the same Visual Studio Code editor with the "app.py" file selected in the Explorer sidebar. The main editor window displays the code for the Flask application:

```

1  from flask import Flask, render_template, request
2  import pickle
3  import numpy as np
4
5  app = Flask(__name__)
6
7
8  with open('credit_model.pkl', 'rb') as f:
9      model = pickle.load(f)
10
11 @app.route('/')
12 def home():
13     return render_template('home.html')
14
15 @app.route('/predict', methods=['POST'])
16 def predict():
17     try:
18
19         gender = int(request.form['gender'])
20         own_car = int(request.form['own_car'])
21         own_realty = int(request.form['own_realty'])
22         income_type = int(request.form['income_type'])
23         education = int(request.form['education'])
24         family_status = int(request.form['family_status'])

```

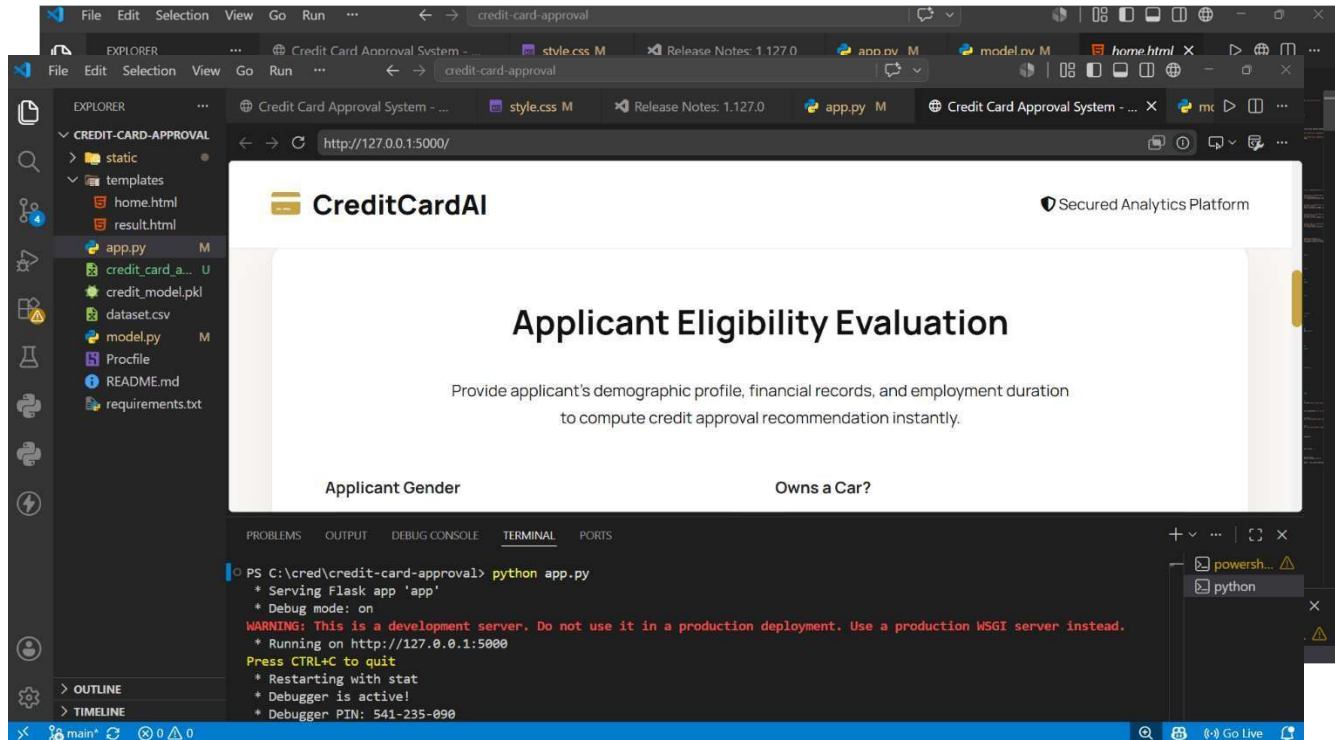
At the bottom of the editor, the "TERMINAL" tab is active, showing the command prompt output:

```

PS C:\cred\credit-card-approval> python app.py
* Debugger is active!
* Debugger PIN: 541-235-090

```

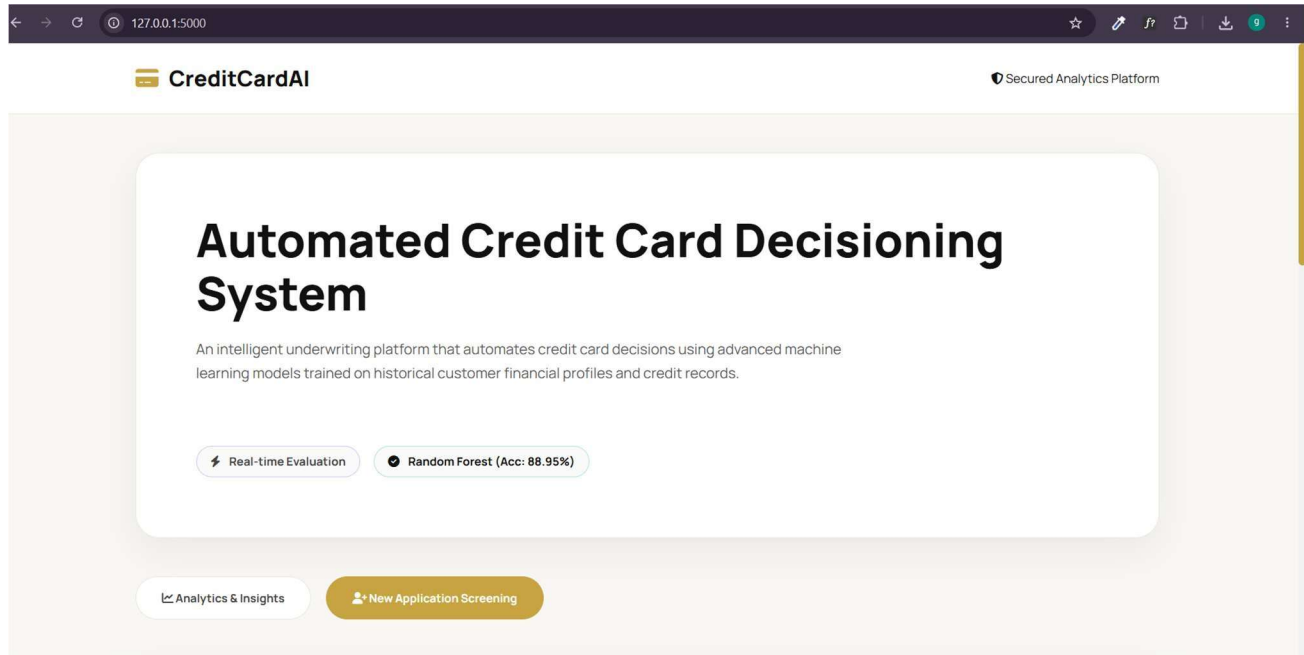
Milestone 4: Frontend Development



Milestone 5: Conclusion

Exploring the Website's Web Pages:

Home / Generator Page:

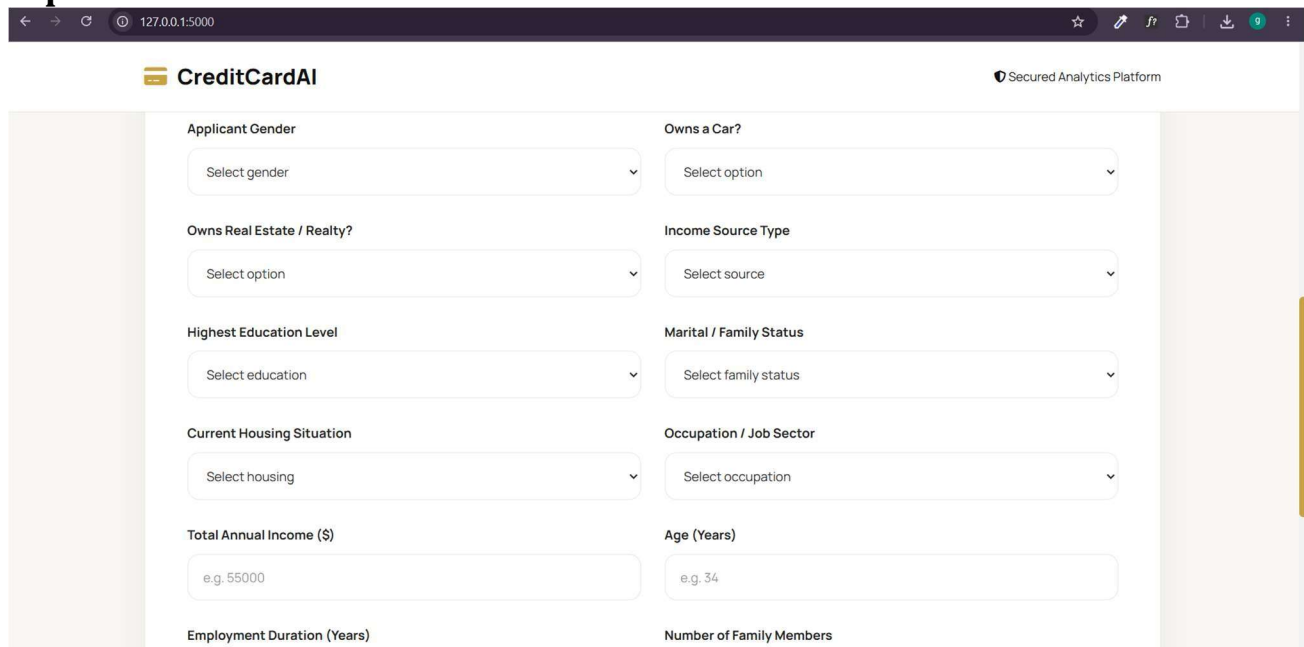


The screenshot shows the home page of the CreditCardAI website. The browser address bar displays "127.0.0.1:5000". The website header includes the "CreditCardAI" logo and a "Secured Analytics Platform" badge. The main content area features a large white box with the title "Automated Credit Card Decisioning System" and a subtitle: "An intelligent underwriting platform that automates credit card decisions using advanced machine learning models trained on historical customer financial profiles and credit records." Below the subtitle are two buttons: "Real-time Evaluation" and "Random Forest (Acc: 88.95%)". At the bottom of the main content area are two buttons: "Analytics & Insights" and "New Application Screening".

Description:

Displays the project title and provides a form for entering applicant information required for credit card approval prediction.

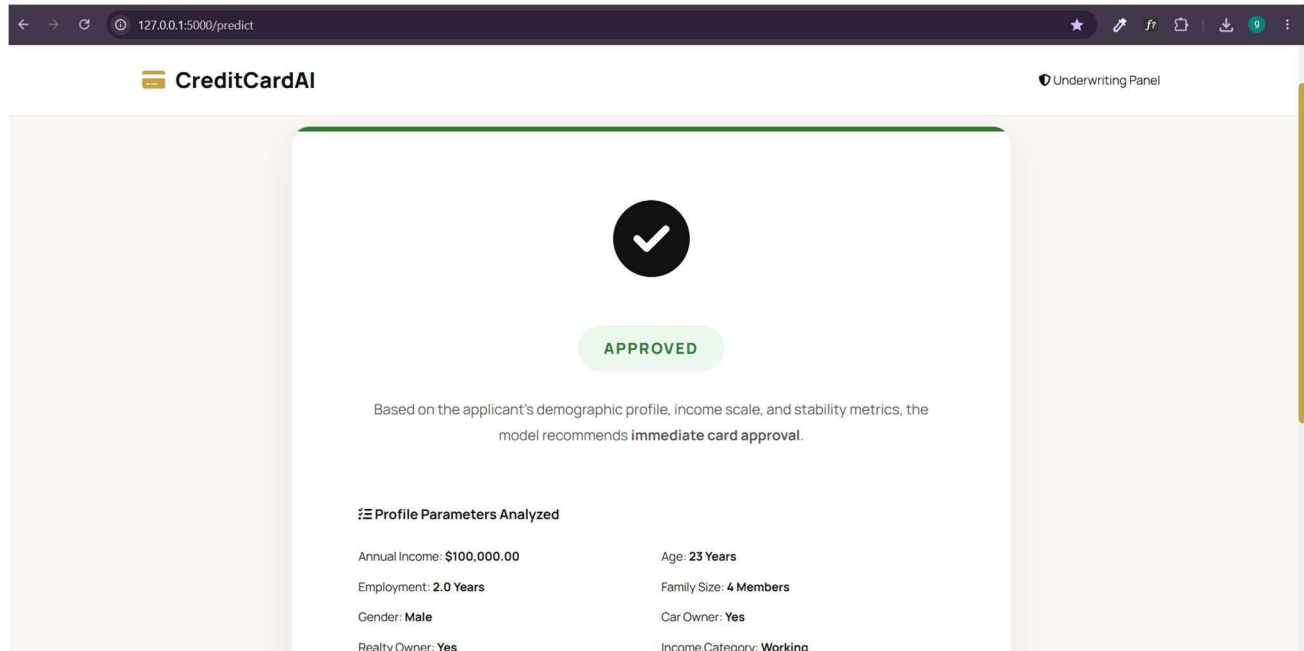
Input Section:



The screenshot shows the input section of the CreditCardAI website. The browser address bar displays "127.0.0.1:5000". The website header includes the "CreditCardAI" logo and a "Secured Analytics Platform" badge. The input section is a form with two columns of fields. The left column contains: "Applicant Gender" (dropdown), "Owns Real Estate / Realty?" (dropdown), "Highest Education Level" (dropdown), "Current Housing Situation" (dropdown), "Total Annual Income (\$)" (text input with placeholder "e.g. 55000"), and "Employment Duration (Years)" (text input). The right column contains: "Owns a Car?" (dropdown), "Income Source Type" (dropdown), "Marital / Family Status" (dropdown), "Occupation / Job Sector" (dropdown), "Age (Years)" (text input with placeholder "e.g. 34"), and "Number of Family Members" (text input).

Allows users to enter financial and demographic details such as income, employment status, family members, education level, housing type, and credit history.

Results Section:



Displays whether the credit card application is Approved or Rejected along with the prediction generated by the machine learning model.

Future Scope

- Improve prediction accuracy using larger datasets.
- Integrate real-time credit score APIs.
- Add secure user authentication.
- Develop Android and iOS applications.
- Deploy on scalable cloud infrastructure.
- Support batch prediction for multiple applicants.

Conclusion:

The Credit Card Approval Prediction system demonstrates how machine learning can automate the credit card approval process by providing fast, accurate, and reliable predictions. By comparing multiple classification algorithms and deploying the best-performing model through Flask and IBM Watson Machine Learning, the project minimizes manual effort, improves decision-making, and enhances customer satisfaction.